

School Notes

Working on this yourself — a guide to the codebase

Next.js · Supabase · TypeScript · ~14,000 lines across 96 files

A map of the codebase and the order I'd learn it in. Written for someone who can already program but hasn't built a web app.

Roughly 14,000 lines, 96 files. Small enough to understand completely.

A printable copy lives at [docs/School-Notes-Architecture.pdf](#), regenerated from this file with `cd web && npm run docs:pdf`.

The one concept that unlocks everything

Next.js runs your code in **two different places**, and almost every confusing error comes from mixing them up.

Server Components (the default) run on the server. They can read the database directly, hold secrets, and `await` things at the top level. They cannot use `useState`, `onClick`, or anything interactive.

Client Components (marked `'use client'` at the top of the file) run in the browser. They can be interactive. They cannot touch the database directly or see secrets.

Look at [web/src/app/notes/page.tsx](#) — a server component. It queries the database and renders a list. Now look at [web/src/components/notes/NoteEditor.tsx](#) — `'use client'` at the top, because it has to respond to typing.

The pattern throughout: **a server component fetches data and passes it as props to a client component that handles interaction.**

How a change gets saved

Client components can't write to the database, so they call a **Server Action** — a function marked `'use server'` that runs on the server but is called like a normal function from the browser.

```
graph LR
    A["NoteEditor.tsx  
(browser)"] -- calls --> B["updateNote()  
(server)"]
    B -- writes --> C["Postgres"]
```

`updateNote` lives in [web/src/app/actions/notes.ts](#). Read that file — it's the clearest example of the whole pattern: validate input, write, then `revalidatePath` so pages re-render with the new data.

What's where

supabase/migrations/	The database. Read these first – everything else is a view onto this.
web/src/app/	Pages and API routes. Folder name = URL.
page.tsx	→ /
notes/page.tsx	→ /notes
notes/[id]/page.tsx	→ /notes/<anything>
api/*/route.ts	→ API endpoints
actions/	Server Actions – how the browser writes data
web/src/components/	Reusable UI. Mostly 'use client'.
web/src/lib/	Logic with no UI. The interesting parts:
canvas/	Canvas import (REST API + .ics parser)
google/	Calendar sync
notion/	Notion sync
calendar/agenda.ts	Merges classes + deadlines + events into one view
ink/strokes.ts	Handwriting maths
supabase/	Database clients

Where to start reading: `supabase/migrations/`, then `web/src/lib/util/dates.ts` (small, pure, well-tested), then `web/src/app/page.tsx` (the dashboard), then `web/src/app/actions/assignments.ts`.

The skills, in the order they pay off

1. TypeScript — a few days. If you know C++ types, this is familiar: `string | null` is a union, `?` means optional, `as` is a cast you should distrust. You do not need generics or decorators to work here.

2. React — the biggest single investment, about a week to be useful. Only three ideas matter for this codebase: components are functions returning markup; `useState` holds values that change; props pass data downward. Skip class components and Redux entirely — neither appears here.

3. SQL and Postgres — you already have the schema to read. The specific thing worth understanding deeply is **row-level security**: every table has a policy saying `user_id = auth.uid()`, which is why a bug in the app still can't leak another user's data. See [supabase/migrations/20260819000004_reminders_rls.sql](#).

4. Next.js App Router — server vs client, above. The bundled docs in `web/node_modules/next/dist/docs/` match your installed version exactly, which web tutorials often won't.

5. Tailwind CSS — not really a skill, a lookup table. `p-4` is padding, `flex` is `display:flex`. Copy an existing element's classes and adjust.

What you can ignore for now: the Google/Notion/Canvas integrations, the service worker, and the ink stroke maths. They're self-contained — you can change everything else without touching them.

Working safely

Run this before you commit anything. It takes about a minute:

```
cd web
npm run typecheck    # catches most mistakes before the browser does
npm run lint
npm test             # 93 unit tests
npm run build        # what Vercel will do
```

Two more, when you've touched the database or something risky:

```
npm run db:check     # applies migrations to a throwaway Postgres
npm run smoke        # 48 checks against the live project
```

Git is your undo button. Before experimenting:

```
git checkout -b trying-something
```

If it goes wrong, `git checkout main` and nothing is lost. If it goes right, commit and merge. Never edit `main` directly for anything non-trivial.

Deploying is `cd web && npx vercel deploy --prod --yes`, but pushing to GitHub is the safer habit — the code is backed up either way.

Exercises, in increasing difficulty

Each one is real and useful. Do them in order.

1. Change something visual. Add a colour to the course palette in [web/src/components/CourseManager.tsx](#) (`PALETTE`). One line. Confirms your setup works.

2. Change a default. Reminders fire 1 day and 2 hours before a deadline. That default lives in the database — `default_reminder_offsets` in [supabase/migrations/20260819000001_core.sql](#). Changing it for yourself is one SQL `update` on your `profiles` row. Teaches you that some behaviour lives in the database, not the code.

- 3. Add a field to a form.** Give courses a "credits" field — the column already exists in the `courses` table, unused. You'll touch: the form fields, the zod schema in `web/src/app/actions/courses.ts`, and the display. This is the single most representative task in the codebase; do it and most other changes become obvious.
 - 4. Write a test.** Add a case to `web/src/lib/util/dates.test.ts` — say, what `relativeDue` returns for something due in exactly one minute. Run `npm test`. Get it failing first, then make it pass.
 - 5. Add a database column.** Give assignments a `where_submitted` note. Write a new migration file, apply it, add the field to `db.ts`, then surface it in the assignment editor. Now you've touched every layer.
 - 6. Add a page.** A "This week" summary at `/week`. Copy the structure of `web/src/app/calendar/page.tsx` and simplify.
-

When something breaks

- **Red squiggles in the editor** — usually a real bug. `npm run typecheck` says the same thing with more context.
 - **"X is not a function" in the browser** — often a server component trying to do something interactive. Does the file need `'use client'`?
 - **Data doesn't refresh after saving** — the action is missing a `revalidatePath`, or the component needs `router.refresh()`.
 - **Empty list that should have data** — check row-level security. Query it in the Supabase SQL editor, which bypasses RLS; if the row is there, the policy or the `user_id` is wrong.
 - **Works locally, breaks deployed** — an environment variable missing in Vercel. `.env.local` is not deployed; those values are set separately.
-

Honest advice

Read code before writing it. The tests are the best documentation here —

`web/src/lib/calendar/agenda.test.ts` explains the calendar's behaviour better than the implementation does.

Change one thing at a time and run it. A week of small working changes beats a weekend of a large broken one.

You don't need to understand all 14,000 lines. You need to understand the ~500 around whatever you're changing.